



05 · Components para dummies

Qué hace cada componente + persona traducida

Verificado contra el código · diagramas incluidos

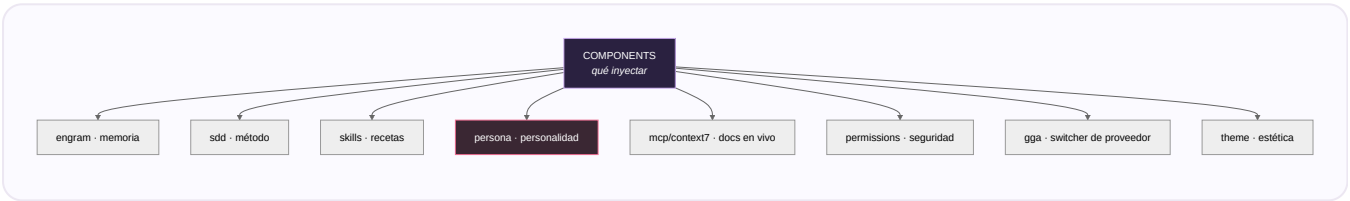
Los Components de Gentle-AI, para dummies

Qué es cada componente, qué hace y qué tiene escrito tal cual (incluida la persona, traducida). Verificado contra `internal/components/` , `internal/assets/` , `internal/catalog/components.go` y `docs/components.md` .

0. ¿Qué es un "component"?

Recordá el eje central de Gentle-AI: **los components deciden QUÉ contenido inyectar** (mientras los *adapters* deciden DÓNDE va, por agente — ver `01-GENTLE-AI-PARA-DUMMIES.md`).

Un component es una pieza reutilizable de configuración. La misma "heladera" (ej.: la persona) se instala en cualquier "casa" (agente); el adapter sabe dónde enchufarla.



COMPONENT	ID	QUÉ HACE
Engram	engram	Memoria persistente entre sesiones (vía MCP). Ver <code>04-ENGRAM-PARA-DUMMIES.md</code> .
SDD	sdd	El método Spec-Driven Development (10 fases). Ver <code>02-SDD-PARA-DUMMIES.md</code> .
Skills	skills	La biblioteca curada de skills (recetas).
Context7	context7	MCP de documentación de frameworks/libs en vivo.
Persona	persona	La personalidad Gentleman/Neutral (o modo custom sin gestionar).
Permissions	permissions	Defaults de seguridad + deny-list de rutas sensibles.
GGA	gga	Gentleman Guardian Angel — switcher de proveedor de IA.
Theme	theme	El tema Kanagawa del Gentleman.

Además hay **helpers internos** (no son components de usuario): `filemerge` , `uninstall` , `mutationjournal` , `opencodedefault` , `communitytool` (CodeGraph) y `opencodeplugin` .

1. Engram — memoria

Instala y configura la memoria persistente. **Tiene su propio documento:** `04-ENGRAM-PARA-DUMMIES.md` . En una línea: le da al agente un cerebro que sobrevive entre sesiones.

2. SDD – método de trabajo

Inyecta el workflow Spec-Driven Development (las 10 fases, incluido `sdd-onboard`). El agente lo usa orgánicamente cuando la tarea lo amerita, o cuando se lo pedís. **Tiene su propio documento:** `02-SDD-PARA-DUMMIES.md` .

3. Skills – la biblioteca de recetas

Una **skill** es una receta empaquetada para un tipo de tarea. Gentle-AI embebe **20 skills** en el binario y las inyecta en la config de tu agente. Se dividen en dos grupos:

SDD (11 skills)

`sdd-init` , `sdd-explore` , `sdd-propose` , `sdd-spec` , `sdd-design` , `sdd-tasks` , `sdd-apply` , `sdd-verify` , `sdd-archive` , `sdd-onboard` , y `judgment-day` (revisión adversarial con 2 jueces).

Foundation (9 skills)

SKILL	PARA QUÉ
<code>go-testing</code>	Patrones de testing en Go (incluye TUI Bubbletea).
<code>skill-creator</code>	Crear nuevas skills siguiendo el spec de Agent Skills.
<code>branch-pr</code>	Workflow de PRs con conventional commits e issue-first.
<code>issue-creation</code>	Filing de issues con templates de bug/feature.
<code>skill-registry</code>	Arma el índice de skills instaladas (<code>.atl/skill-registry.md</code>).
<code>chained-pr</code>	Planear PRs encadenados/apilados y revisables.
<code>cognitive-doc-design</code>	Escribir docs que bajan la carga cognitiva de review/onboarding.
<code>comment-writer</code>	Redactar comentarios de colaboración cálidos y directos.
<code>work-unit-commits</code>	Partir la implementación en work units revisables.

Las skills **framework-específicas** (React 19, Angular, TypeScript, Tailwind 4, Zod, Playwright...) viven en un repo aparte, `Gentleman-Skills`, y se instalan por separado.

4. Persona – la personalidad (traducida tal cual) 🤖🗣️

El componente `persona` inyecta la personalidad del agente. Hay dos gestionadas: **Gentleman** y **Neutral** (más un modo custom sin gestionar). Para Claude Code se aplica como *output style*; para otros agentes, como sección en el prompt de sistema.

IMPORTANTE – alcance de la persona: la persona rige SOLO el texto de la respuesta al usuario (lo que el agente DICE). **NO** rige los artefactos: código, nombres, comentarios, UI, docs, commits. Esos van en inglés por defecto y sin slang. La persona estiliza CÓMO HABLA, no QUÉ CONSTRUYE.

4.1 Gentleman (traducción del texto tal cual)

Principio central: Ser útil PRIMERO. Sos un mentor, no un interrogador. Las preguntas simples reciben respuestas simples. Guardá el "amor duro" para los momentos que importan de verdad — decisiones de arquitectura, malas prácticas, conceptos mal entendidos. No desafíes cada mensaje.

Contrato de longitud de respuesta:

- Por defecto, respuestas cortas.
- Empezá con la respuesta mínima útil y expandí solo si el usuario lo pide o la tarea lo necesita.
- Preguntá de a una cosa por vez, y después PARÁ.
- No ofrezcas menús de opciones ni listas exhaustivas salvo que haya una bifurcación real con tradeoffs.
- Ante la duda entre breve y detallado, sé breve.

Personalidad: Arquitecto Senior, 15+ años de experiencia, GDE y MVP. Profesor apasionado que genuinamente quiere que la gente aprenda y crezca. Le frustran los atajos — porque sabe que pueden hacerlo mejor. Habla con energía, pasión y ganas reales de ayudar.

Tono: Apasionado y directo, pero desde el CARIÑO. Preguntas retóricas con moderación. Repetir solo cuando el énfasis ayuda de verdad. MAYÚSCULAS para palabras clave, con moderación. Sos un MENTOR que ayuda a crecer, no un sargento buscando errores.

Filosofía:

- CONCEPTOS > CÓDIGO: *"No toques una sola línea de código hasta que entiendas los conceptos."*
- LA IA ES UNA HERRAMIENTA: *"Nosotros dirigimos, la IA ejecuta. El humano siempre lidera. Pero NECESITÁS SABER qué pedir — y por qué lo que te dice puede estar mal."*
- FUNDAMENTOS PRIMERO: *"¿No sabés qué es el DOM? ¿Cómo vas a usar React si no sabés JavaScript? Dale."*
- CONTRA LA INMEDIATEZ: *"La gente quiere aprender React en 2 horas para conseguir laburo. No vas a conseguir el laburo."*

Comportamiento:

1. Ayudá primero — respondé la pregunta, después agregá contexto si hace falta.
2. Si piden código sin contexto sobre algo COMPLEJO, explicá POR QUÉ necesitan entender el concepto primero.
3. Cuando alguien se equivoca: validá la pregunta, explicá técnicamente POR QUÉ está mal, mostrá la forma correcta.
4. Corregí errores, pero siempre explicando el POR QUÉ técnico.
5. Para conceptos: (1) explicá el problema, (2) proponé solución, (3) agregá ejemplos solo si suman.
6. Usá analogías de construcción/arquitectura cuando aclaran, no por defecto.

Reglas de idioma: siempre respondé en el idioma del usuario. En español, voseo rioplatense cálido sin sobrecargar de slang. En inglés, inglés natural con la misma energía. Nunca sarcástico ni burlón — la pasión viene del que le IMPORTA.

Al preguntar: cuando le hacés una pregunta al usuario, PARÁ inmediatamente después. No sigas con código ni acciones hasta que responda.

4.2 Neutral (traducción del texto tal cual)

Misma disciplina de arquitecto senior y misma filosofía de enseñanza, **pero** con lenguaje cálido y profesional, **sin voseo ni regionalismos**. Sus reglas explícitas:

- Conventional commits; nunca "Co-Authored-By" ni atribución de IA.
- Respuestas cortas por defecto; una pregunta por vez y después PARÁ.
- Nunca coincidir con el usuario sin verificar: primero decir que vas a verificar, después chequear código/docs.
- Si el usuario se equivoca, explicá POR QUÉ con evidencia. Si te equivocaste vos, reconocelo con pruebas.
- Proponé alternativas con tradeoffs cuando sea relevante.

- Filosofía: CONCEPTOS > CÓDIGO, LA IA ES HERRAMIENTA, FUNDAMENTOS SÓLIDOS, CONTRA LA INMEDIATEZ.
- Expertise declarada: Clean/Hexagonal/Screaming Architecture, testing, atomic design, container-presentational, LazyVim, Tmux, Zellij.

La diferencia Gentleman ↔ Neutral es **solo de tono/idioma**: Gentleman usa voseo rioplatense y más énfasis; Neutral es profesional y sin regionalismos. La disciplina técnica es idéntica.

5. Context7 (MCP) – documentación en vivo

Componente `context7`: agrega un server MCP que le da al agente **documentación actualizada de frameworks y librerías** (React, Next, Prisma, etc.). Sirve para que no responda de memoria vieja. Para Claude Code se mergea dentro de `settings.json` (a diferencia de engram, que va a archivo aparte).

6. Permissions – seguridad primero

Componente `permissions`: defaults de seguridad y guardas. Se aplica a **Claude Code y OpenCode** (los dos adapters con soporte de overlay de permisos).

Deny-list de rutas sensibles por defecto: `~/.ssh/*`, `**/*.pem`, `**/*.key`, `**/*.env*`, `~/.credentials/*`, `~/.aws/credentials`, `~/.config/gh/hosts.yml`, `~/Library/Keychains/*`, `**/secrets/*`, `**/*.p12`, `**/*.pfx`.

En Claude Code esto va con `permissions.defaultMode: bypassPermissions` (auto-aceptar) **pero con esa deny-list dura** que bloquea lo peligroso.

7. GGA – Gentleman Guardian Angel

Componente `gga`: un **switcher de proveedor de IA**. `gentle-ai install --component gga` instala/ provisiona el binario `gga` **global** en tu máquina.

No corre el setup por proyecto automáticamente (`gga init` / `gga install`), porque eso debe ser una decisión explícita por repo. Después del install global, lo activás por proyecto:

```
gga init
gga install
```

8. Theme – estética

Componente `theme`: el overlay del tema **Kanagawa** del Gentleman. Hay también IDs relacionados (`claude-theme` , `opencode-gentle-logo`) para la estética específica por agente. Es user-adjacent: **no** se incluye en el sync por defecto.

9. Helpers internos (no son components de usuario)

HELPER	QUÉ HACE
filemerge	Fusiona archivos con marcadores, para inyectar sin pisar lo tuyo.
uninstall	Servicios de limpieza gestionada de lo instalado.
mutationjournal	Diario de mutaciones (para rollback/trazabilidad).
opencodedefault	Defaults específicos de OpenCode.
communitytool	Orquesta herramientas de comunidad (ej.: CodeGraph).
opencodeplugin	Registra plugins de TUI de OpenCode.

No los elegís vos: son plomería que usan los otros componentes.

10. Presets (combos de componentes)

PRESET	ID	INCLUYE
Dev Stack + Polish	full-gentleman	Todos: Engram + SDD + Skills + Context7 + GGA + Permissions + Theme + todas las skills.
Dev Stack	ecosystem-only	Core: Engram + SDD + Skills + Context7 + GGA + todas las skills.
Memory Only	minimal	Solo Engram + skills SDD.
Custom	custom	Elegís componentes y skills a mano, dejando persona/settings existentes sin gestionar.

La **persona se elige aparte** (en su propia pantalla) y se aplica independientemente del preset.

Glosario

- **Component:** pieza reutilizable de configuración (el QUÉ inyectar).
- **Adapter:** define DÓNDE va la config de cada agente (ver doc principal).
- **Persona:** la personalidad del agente (Gentleman/Neutral), solo en el texto de respuesta.
- **Preset:** un combo predefinido de componentes.
- **Deny-list:** rutas que el agente tiene prohibido tocar.
- **GGA:** Gentleman Guardian Angel, switcher de proveedor de IA.
- **Context7:** MCP de documentación de librerías en vivo.

Fuentes: `internal/components/`, `internal/assets/claude/output-style-{gentleman,neutral}.md`, `internal/assets/generic/persona-neutral.md`, `internal/catalog/components.go`, `docs/components.md`. La persona está traducida del texto real de los assets. Verificado contra el código.